

DSAA Lab 07

Hashing Technique

Areeba (TCT-004)

Thar Institute of Engineering Sciences and Technology (TIEST)

CT-159: Data Structure Algorithms and Applications (Pr)

Engr. Aneel Kumar

October 30, 2025

Exercise

Q1: The keys 12, 18, 13, 2, 3, 23, 5 and 15 are inserted into an initially empty hash table of length 10 using open addressing with hash function $h(k) = k \bmod 10$ and linear probing. What is the resultant hash table?

Answer:

Hash function: $h(k) = k \bmod 10$

Collision resolution: linear probing

Insertions (in order): 12, 18, 13, 2, 3, 23, 5, 15

Step-by-step :

- $12 \rightarrow h(12)=2 \rightarrow$ place at index 2.
- $18 \rightarrow h(18)=8 \rightarrow$ place at index 8.
- $13 \rightarrow h(13)=3 \rightarrow$ place at index 3.
- $2 \rightarrow h(2)=2 \rightarrow$ index 2 occupied (12), probe 3 occupied (13), probe 4 empty $\rightarrow$ place 2 at index 4.
- $3 \rightarrow h(3)=3 \rightarrow$ 3,4 occupied $\rightarrow$ probe 5 empty $\rightarrow$ place 3 at index 5.
- $23 \rightarrow h(23)=3 \rightarrow$ 3,4,5 occupied $\rightarrow$ probe 6 empty $\rightarrow$ place 23 at index 6.
- $5 \rightarrow h(5)=5 \rightarrow$ 5,6 occupied $\rightarrow$ probe 7 empty $\rightarrow$ place 5 at index 7.
- $15 \rightarrow h(15)=5 \rightarrow$ 5,6,7 occupied $\rightarrow$ probe 8 occupied (18) $\rightarrow$ probe 9 empty $\rightarrow$ place 15 at index 9.

0	-
1	-
2	12
3	13
4	2
5	3
6	23
7	5
8	18
9	15

Q2: The keys 11, 9, 7, 8 are inserted into an initially empty hash table of length 10 using open addressing with hash function $h(k) = k \bmod 5$ and linear probing.

Answer:

Hash function: $h(k) = k \bmod 5$

Collision resolution: linear probing

Table length: 10

Insertions: 11, 9, 7, 8

Step-by-step:

- $11 \rightarrow h(11)=1 \rightarrow$ place at index 1.
- $9 \rightarrow h(9)=4 \rightarrow$ place at index 4.
- $7 \rightarrow h(7)=2 \rightarrow$ place at index 2.
- $8 \rightarrow h(8)=3 \rightarrow$ place at index 3.

0	-
1	11
2	7
3	8
4	9
5	-
6	-
7	-
8	-
9	-

Q3: Implement a hashing algorithm using Linear Probing.

Answer:

```
import java.util.Scanner;

public class LinearProbingHash {
    int size;
    int[] table;

    LinearProbingHash(int size) {
        this.size = size;
        table = new int[size];
        for (int i = 0; i < size; i++)
            table[i] = -1; // -1 means empty
    }

    void insert(int key) {
        int index = key % size;
        while (table[index] != -1) { // if occupied, move to next
            index = (index + 1) % size;
        }
        table[index] = key;
    }

    void display() {
        for (int i = 0; i < size; i++)
            System.out.println(i + " : " + (table[i] == -1 ? "-" : table[i]));
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        LinearProbingHash h = new LinearProbingHash(10);
        System.out.println("Enter 5 keys:");
        for (int i = 0; i < 5; i++) {
            int key = sc.nextInt();
            h.insert(key);
        }
        System.out.println("Hash Table:");
        h.display();
    }
}
```

Q4: Implement a hashing algorithm using Double Hashing

Answer:

```
import java.util.Scanner;

public class DoubleHashing {
    int size;
    int[] table;

    DoubleHashing(int size) {
        this.size = size;
        table = new int[size];
        for (int i = 0; i < size; i++)
            table[i] = -1;
    }

    int hash1(int key) {
        return key % size;
    }

    int hash2(int key) {
        return 7 - (key % 7); // secondary hash (should be non-zero)
    }

    void insert(int key) {
        int index = hash1(key);
        int step = hash2(key);
        while (table[index] != -1) {
            index = (index + step) % size; // jump by step size
        }
        table[index] = key;
    }

    void display() {
        for (int i = 0; i < size; i++)
            System.out.println(i + " : " + (table[i] == -1 ? "-" : table[i]));
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        DoubleHashing h = new DoubleHashing(10);
        System.out.println("Enter 5 keys:");
        for (int i = 0; i < 5; i++) {
            int key = sc.nextInt();
            h.insert(key);
        }
        System.out.println("Hash Table:");
        h.display();
    }
}
```

